



**Kaunas University of Technology**  
Faculty of Electrical and Electronics Engineering

# **Coffee Leaf Disease Classification Report**

## **Report**

---

**Gerson David Pinto Chica**  
Project Author

**Dr. Arūnas Lipnickas**  
Supervisor

---

**Kaunas, 2026**

## **Contents**

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>List of Figures</b>                          | <b>3</b>  |
| <b>List of Tables</b>                           | <b>4</b>  |
| <b>1 Abstract</b>                               | <b>5</b>  |
| <b>2 Introduction</b>                           | <b>5</b>  |
| <b>3 Dataset Description</b>                    | <b>5</b>  |
| <b>4 Preprocessing and Experimental Setup</b>   | <b>6</b>  |
| <b>5 Model 1: MobileNetV2 Transfer Learning</b> | <b>7</b>  |
| <b>6 Model 2: Simple CNN</b>                    | <b>7</b>  |
| <b>7 Results</b>                                | <b>8</b>  |
| <b>8 Model Comparison and Discussion</b>        | <b>9</b>  |
| <b>9 Limitations and Improvements</b>           | <b>11</b> |
| <b>10 Conclusion</b>                            | <b>12</b> |
| <b>References</b>                               | <b>13</b> |

**List of Figures**

**Fig. 1.** Count of images per class and split after image cleansing. . . . . 6

**Fig. 2.** Training and validation accuracy and loss curves for MobileNetV2 with frozen base model. . . . . 8

**Fig. 3.** Training and validation accuracy and loss curves for Simple CNN model. . . . . 9

**Fig. 4.** Confusion matrix of MobileNetV2 on the test set. . . . . 10

**Fig. 5.** Confusion matrix of Simple CNN on the test set. . . . . 11

**List of Tables**

**Table 1.** Count of usable images after image cleansing. . . . . 6

**Table 2.** Summary of the MobileNetV2 transfer learning model. . . . . 7

**Table 3.** Summary of the Simple CNN model. . . . . 8

**Table 4.** General comparison between both models based on the experimental outputs. . . 9

**Table 5.** Precision, recall, and F1-score per class in the classification reports. . . . . 10

## **1. Abstract**

In this report, two image classification models trained on the same coffee leaf disease dataset were compared. The objective of this study was to classify images of coffee leaves into one of three classes – healthy leaves, coffee leaf rust, and Phoma disease. The models tested in this experiment were a MobileNetV2 transfer learning model and a Simple CNN. Both models used the same training, validation, and testing folders, the same batch size, and input images of 224 x 224 pixels. The three output classes for both models were the same as well. The Simple CNN model performed better in terms of test accuracy, achieving 95.88%, compared to 94.43% in the case of the MobileNetV2 model. However, in terms of the training time, the Simple CNN model needed 82 minutes for training while MobileNetV2 needed only 16 minutes for the frozen-base model run.

## **2. Introduction**

Coffee leaf diseases may decrease crop productivity and quality. Machine vision-based computer models can assist farmers with automated disease detection by learning from labeled images of healthy and diseased leaves. In the current project, the task was not only to build a model that performed classification accurately, but also to compare two methods and analyze the differences between the chosen models. Such analysis was useful since the model with the best accuracy was not necessarily the most efficient one to deploy. In this paper, the criteria for comparison included such parameters as accuracy, loss, performance per class, the number of model parameters, preprocessing, limitations and possible problems with the dataset.

Two models discussed in this experiment belonged to two widely-used types of image classification techniques. The first one used transfer learning based on pre-trained MobileNetV2. Only a classification head was trained while the remaining feature extraction part of the model was kept frozen. The second approach used a custom CNN architecture which learned convolutional filters during the training process using the dataset. Both of these models were applied depending on the conditions: transfer learning in case there was a lack of labeled data and vice versa.

## **3. Dataset Description**

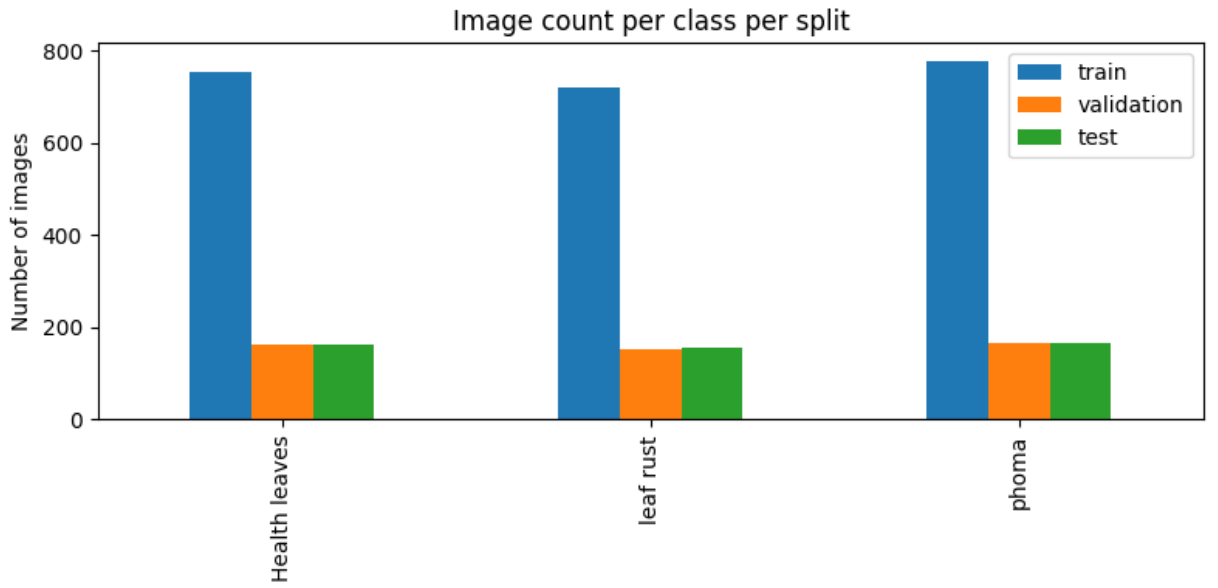
The external dataset used in this experiment was “A Machine Learning Dataset for Classification of Common Coffee Leaf Diseases in Uganda.” The dataset was posted on Mendeley Data with DOI: 10.17632/k36wnd6knb.1 [1]. According to the source page, the dataset contained a set of labeled images of coffee leaf diseases collected under various lighting conditions on farms in Uganda. These images were classified into three classes: healthy leaves, leaf rust, and Phoma disease [1]. Images were stored as JPEG files of 256x256 pixels, whereas the experiment loaded the images at 224x224 pixels prior to the training [1].

An important point about reproducibility of this dataset was noted. According to the description, the

Mendeley source provided 3,312 images, but the local dataset contained a slightly different number of files before cleanup. Corrupted files were then cleaned, and 2,252 training images, 483 validation images, and 485 test images were used for the model training in total of 3,220 images. In this report, the external source was used for documenting the dataset origin, and model results were obtained with the exact numbers provided by the experimental outputs.

**Table 1:** Count of usable images after image cleansing.

| Class          | Training images | Validation images | Test images | Total usable images |
|----------------|-----------------|-------------------|-------------|---------------------|
| Healthy leaves | 754             | 163               | 162         | 1,079               |
| Leaf rust      | 721             | 154               | 156         | 1,031               |
| Phoma          | 777             | 166               | 167         | 1,110               |
| Total          | 2,252           | 483               | 485         | 3,220               |



**Fig. 1.** Count of images per class and split after image cleansing.

#### 4. Preprocessing and Experimental Setup

The images were preprocessed and prepared for training in the same way for both models. The images were read from folders and scaled to 224 x 224 pixels. They were batched with batch size 32 and transformed to categorical labels. The training set was augmented with random rotations, width shifts, height shifts, horizontal flips, and zooming. Validation and test data were not augmented; they were only rescaled.

Models were compiled with categorical cross-entropy as a loss function and Adam optimizer with the initial learning rate of 0.001. The early stopping strategy, reduce learning rate on plateau, and save best model were used during model training. As documented by Keras, EarlyStopping was a callback that stopped model training when no improvement in the monitored quantity was observed. Similarly, ReduceLROnPlateau was a callback that decreased learning rate upon a lack of improvement during training [2, 3]. Finally, the classification reports were generated with scikit-learn using its classification\_report function [4].

A methodology limitation was related to the MobileNetV2 preprocessing pipeline. According to the MobileNetV2 documentation in Keras, rescaling to [0..1] interval was not recommended. The documentation recommended using the MobileNetV2 preprocessing function to rescale inputs to [-1..1] interval. Even though the model achieved very good results, its preprocessing pipeline was not fully aligned with the recommended transfer-learning preprocessing method.

## 5. Model 1: MobileNetV2 Transfer Learning

Model 1 applied MobileNetV2 for feature extraction with transfer learning. Specifically, MobileNetV2 was proposed as an efficient mobile network with inverted residuals and linear bottlenecks [5]. Similarly, the Keras application guide mentioned that MobileNetV2 applied inverted residual blocks with bottlenecks and supported ImageNet pre-trained weights [6].

The MobileNetV2 base was loaded with ImageNet weights, the original top classifier was removed, and the base was frozen. The new classification head contained global average pooling, a 0.3 rate dropout, and a dense softmax classifier with three output classes. Thus, the workflow matched the typical transfer learning pipeline as follows: loading pre-trained base model, freezing the base model, creating and stacking the new model on top, and then training the classifier on the target data [7].

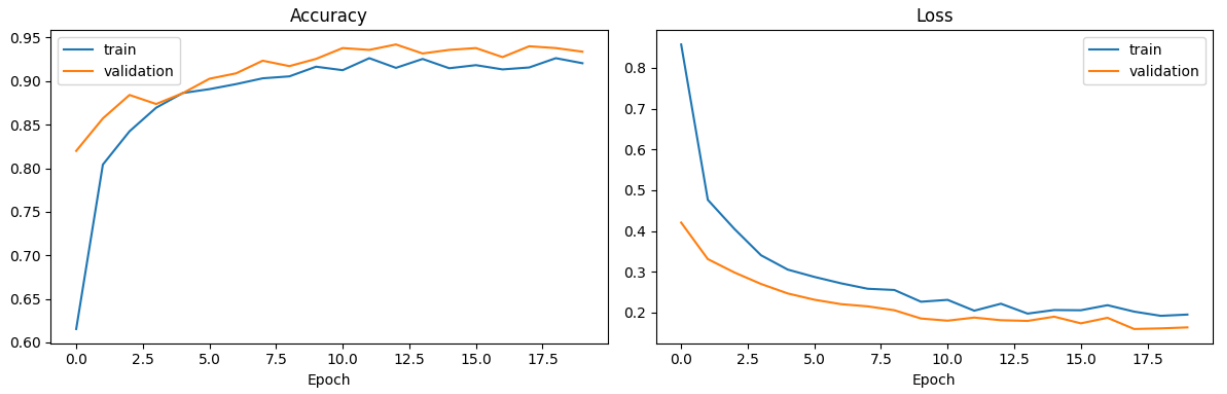
**Table 2:** Summary of the MobileNetV2 transfer learning model.

| Component                               | Description                                      |
|-----------------------------------------|--------------------------------------------------|
| Base model                              | MobileNetV2; include_top=False; ImageNet weights |
| Training status of the base model       | Frozen during the main training run              |
| Pooling layer                           | GlobalAveragePooling2D                           |
| Dropout                                 | 0.3                                              |
| Classifier output layer                 | Dense layer with 3 softmax outputs               |
| Total parameters                        | 2,261,827                                        |
| Trainable parameters                    | 3,843                                            |
| Non-trainable parameters                | 2,257,984                                        |
| Number of epochs with frozen base model | 20 epochs                                        |

Additionally, an optional fine-tuning stage was included where the top layers of MobileNetV2 were unfrozen and trained with a reduced learning rate. The results comparison included the primary MobileNetV2 model evaluation score of 94.43% test accuracy.

## 6. Model 2: Simple CNN

Model 2 was a custom convolutional neural network trained from scratch. In contrast to Model 1, Model 2 did not use pre-trained ImageNet weights. The architecture consisted of several convolutional blocks with Conv2D layers, batch normalization, max pooling, and dropout. The last classifier contained dense layers with three class softmax outputs. Therefore, this design enabled Model 2 to



**Fig. 2.** Training and validation accuracy and loss curves for MobileNetV2 with frozen base model.

extract features from the coffee leaf images directly. Still, the training time was longer compared to MobileNetV2 since all trainable filters needed to be learned from scratch using the project dataset.

**Table 3:** Summary of the Simple CNN model.

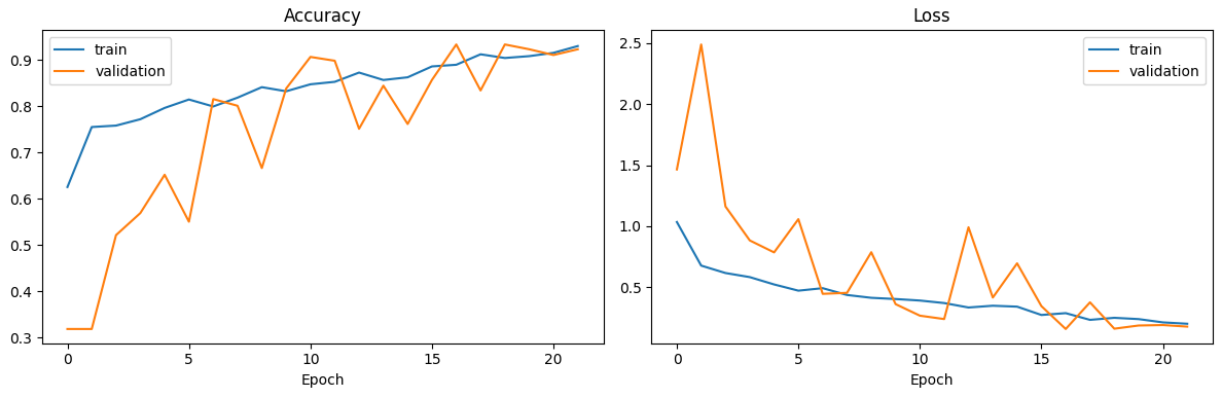
| Component                               | Description                                         |
|-----------------------------------------|-----------------------------------------------------|
| Architecture type                       | Custom CNN trained from scratch                     |
| Input image size                        | 224 x 224 x 3                                       |
| Layers                                  | Conv2D, BatchNormalization, MaxPooling2D, Dropout   |
| Classifier output layer                 | Dense layer with 3 softmax outputs                  |
| Total number of parameters              | 1,442,851                                           |
| Training status of all model parameters | All model weights trained from the coffee leaf data |
| Number of epochs completed              | 22 epochs                                           |
| Total training time                     | 4,919.10 seconds / 81.99 minutes                    |

The training curves demonstrated gradual improvement and slightly unstable validation loss in the initial training epochs. This was expected for a CNN model trained from scratch due to randomly initialized network weights. Once the network learned useful visual filters from the input images, validation accuracy increased, and the model restored the best performance weights according to the callback settings.

## 7. Results

The same hold-out test set of 485 images was used for both models. This was an important criterion for comparability since both models' performance was measured against the same image distribution and class labels. The Simple CNN model yielded the highest overall test accuracy and lowest test loss value in this run.





**Fig. 3.** Training and validation accuracy and loss curves for Simple CNN model.

**Table 4:** General comparison between both models based on the experimental outputs.

| Metric                           | MobileNetV2 transfer learning | Simple CNN                       | Best result                            |
|----------------------------------|-------------------------------|----------------------------------|----------------------------------------|
| Test accuracy                    | 94.43%                        | 95.88%                           | Simple CNN                             |
| Test loss                        | 0.1630                        | 0.1158                           | Simple CNN                             |
| Total parameters                 | 2,261,827                     | 1,442,851                        | Simple CNN smaller total model         |
| Trainable parameters in main run | 3,843                         | 1,442,851                        | MobileNetV2 fewer trainable parameters |
| Training time                    | 969 seconds / 16.15 minutes   | 4,919.10 seconds / 81.99 minutes | MobileNetV2 faster training            |
| Model size in notebook summary   | 8.63 MB                       | 5.50 MB                          | Simple CNN smaller file/model summary  |
| Training strategy                | Transfer learning             | From scratch                     | Depending on objective                 |

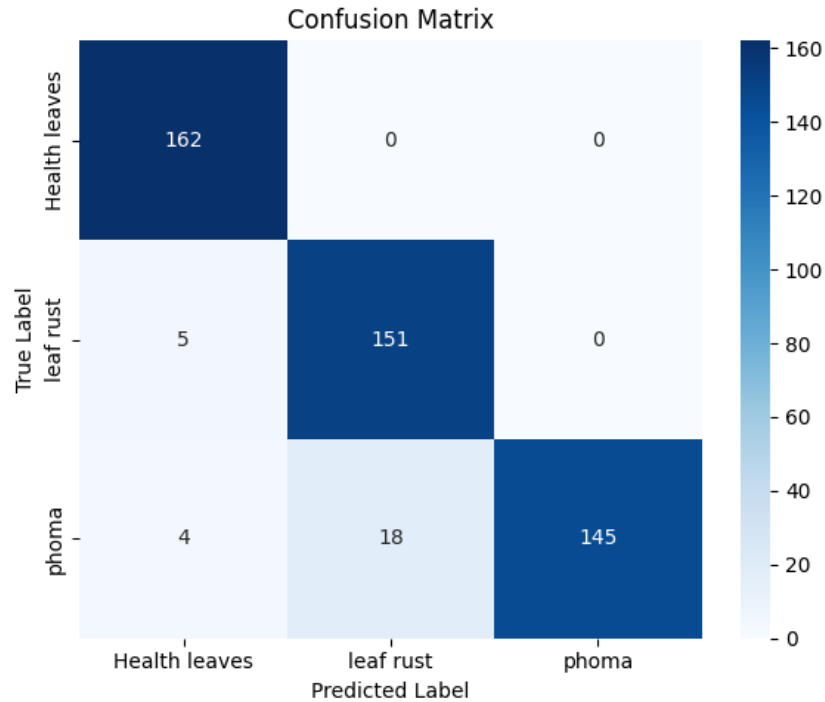
## 8. Model Comparison and Discussion

The Simple CNN yielded the highest numerical score in this run, achieving a test accuracy of 95.88%, compared to 94.43% by MobileNetV2. The gap between the two was 1.45 percentage points. Furthermore, the Simple CNN model also improved the recall value of the Phoma label from 0.87 to 0.92. This was an improvement of 0.05, which indicated that Simple CNN identified fewer false-negative Phoma instances than MobileNetV2. This was relevant since disease identification aimed to minimize false negatives.

However, Simple CNN was not automatically the superior model. The Simple CNN required approximately 82 minutes for training, compared to around 16 minutes for MobileNetV2. This was because only 3,843 parameters were trainable in the main run of MobileNetV2, compared to 1.44 million for the Simple CNN model. Training time was an important criterion when quick model testing was

**Table 5:** Precision, recall, and F1-score per class in the classification reports.

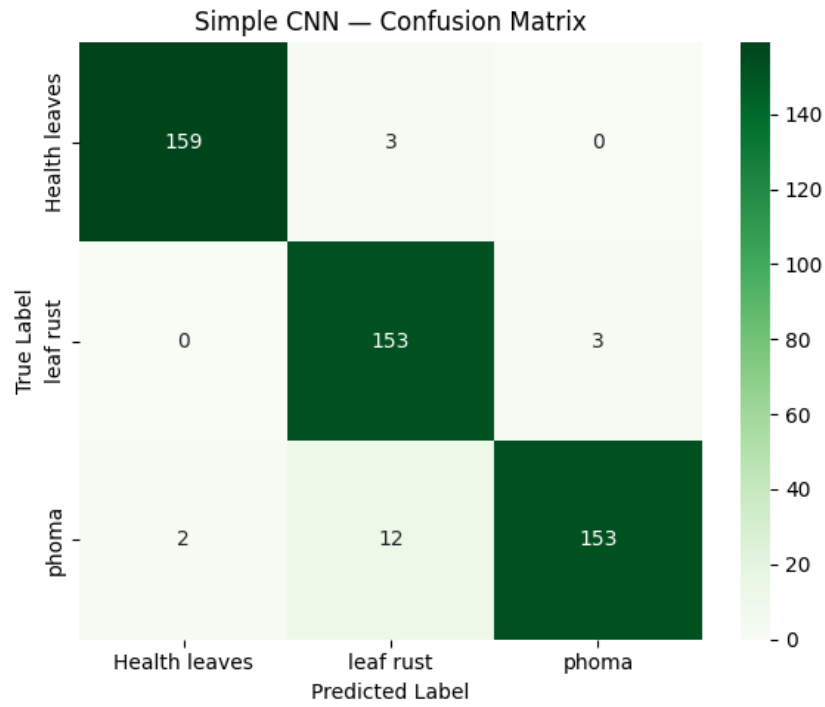
| Class            | MobileNetV2 precision | MobileNetV2 recall | MobileNetV2 F1 | Simple CNN precision | Simple CNN recall | Simple CNN F1 |
|------------------|-----------------------|--------------------|----------------|----------------------|-------------------|---------------|
| Healthy leaves   | 0.95                  | 1.00               | 0.97           | 0.99                 | 0.98              | 0.98          |
| Leaf rust        | 0.89                  | 0.97               | 0.93           | 0.91                 | 0.98              | 0.94          |
| Phoma            | 1.00                  | 0.87               | 0.93           | 0.98                 | 0.92              | 0.95          |
| Macro average    | 0.95                  | 0.95               | 0.94           | 0.96                 | 0.96              | 0.96          |
| Weighted average | 0.95                  | 0.94               | 0.94           | 0.96                 | 0.96              | 0.96          |

**Fig. 4.** Confusion matrix of MobileNetV2 on the test set.

crucial, thus making MobileNetV2 more efficient.

The confusion matrix results revealed that both models could effectively classify healthy leaves, as seen by their high values in the True Positive and True Negative cells. However, MobileNetV2 could not identify all healthy test samples but classified 18 images labeled as Phoma into the leaf rust category. In contrast, the Simple CNN made mistakes when identifying healthy samples but minimized Phoma-to-rust confusion by classifying 12 Phoma samples as leaf rust instead. Thus, Simple CNN seemed to better distinguish the two classes in this run.

Finally, care was needed in interpreting the result since the dataset source mentioned that augmentation techniques were utilized to improve sample diversity [1]. Augmented versions of the same leaves might have been included in each split, leading to an overestimated measurement of accuracy. While random training, validation, and test folders were generated, there was no evidence that the data were



**Fig. 5.** Confusion matrix of Simple CNN on the test set.

split in a manner that ensured each sample group contained similar augmented variants.

## 9. Limitations and Improvements

The requirement for the assignment was fulfilled since both models were trained and tested on the same dataset with coffee leaves. Some limitations affected the scientific nature of the report:

1. The official MobileNetV2 preprocessing pipeline was not applied, since the current rescale = 1/255 was used instead of the preprocess\_input() function.
2. The fine-tuned MobileNetV2 model was not evaluated separately on the test set. Although the fine-tuning block of the MobileNetV2 model was included, there was no evaluation of its results.
3. The code was not run using several random seeds, so the average accuracy and average F1-score were not reported.
4. Possible data leakage was not fully excluded when splitting the datasets into training, validation, and test sets. Augmented images or duplicates could have appeared in other sets besides the test one.
5. The models were not evaluated on an additional dataset or with images collected during the project.
6. The inference time per image and memory consumption on the targeted device were not estimated. Training time alone could not be the determinant for selecting a particular model.

7. An explanation technique such as Grad-CAM was not added to confirm that the models learned features from disease regions rather than background artifacts.

## **10. Conclusion**

This report provided a comparison of two models on a specific dataset containing coffee leaf disease images. The Simple CNN model gave the best test performance results with 95.88% accuracy. MobileNetV2 achieved the second-best test accuracy, with 94.43%. Moreover, MobileNetV2 also showed high F1-scores. It is worth mentioning that the MobileNetV2 model was trained much faster as only the classifier was trained on the pre-trained base model. Therefore, the result did not show that one model was always the best, but that each model had its own advantages. The Simple CNN model gave better results in this notebook run, but MobileNetV2 was more efficient during training.

Both models were used for the comparison. On the other hand, some limitations were also taken into account. Thus, among the technical errors, there were the use of incorrect MobileNetV2 preprocessing, lack of evaluation of the fine-tuned MobileNetV2 model separately, and possibility of data leakage due to image augmentations.

## References

- [1] S. Chelangat, R. Anirwoth, K. N. Mayanja, and A. Sserwadda. A machine learning dataset for classification of common coffee leaf diseases in uganda. Mendeley Data, Version 1, 2025. URL <https://data.mendeley.com/datasets/k36wnd6knb/1>.
- [2] Keras. Earlystopping. Keras Callbacks API documentation, 2026. URL [https://keras.io/api/callbacks/early\\_stopping/](https://keras.io/api/callbacks/early_stopping/).
- [3] Keras. Reducelronplateau. Keras Callbacks API documentation, 2026. URL [https://keras.io/api/callbacks/reduce\\_lr\\_on\\_plateau/](https://keras.io/api/callbacks/reduce_lr_on_plateau/).
- [4] scikit-learn. classification\_report. scikit-learn API documentation, 2026. URL [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification\\_report.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html).
- [5] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. URL [https://openaccess.thecvf.com/content\\_cvpr\\_2018/papers/Sandler\\_MobileNetV2\\_Inverted\\_Residuals\\_CVPR\\_2018\\_paper.pdf](https://openaccess.thecvf.com/content_cvpr_2018/papers/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.pdf).
- [6] Keras. Mobilenet, mobilenetv2, and mobilenetv3. Keras Applications documentation, 2026. URL <https://keras.io/2/api/applications/mobilenet/>.
- [7] Keras. Transfer learning and fine-tuning. Keras Guides, 2026. URL [https://keras.io/guides/transfer\\_learning/](https://keras.io/guides/transfer_learning/).